Understanding Mathematical Concepts via Go Language

Vector

Hyosang Kang (DGIST)

고 언어로 수학적 개념 이해하기

벡터

강효상 (DGIST)

Vector

Point as a Finite Sequence of Real Numbers

$$P = (x_1, x_2, \ldots, x_n)$$

```go
type Map[T1, T2 any] interface {
	Map(T1) T2
}

type Finite interface {
	Len() int
}

type Point interface {
	Element
	Finite
	Map[int, Real]
}
```

Vector

1. Addition

$$\mathbf{v} + \mathbf{w} = \mathbf{w} + \mathbf{v}$$

2. Scalar Multiplication

$$a \cdot \mathbf{v} = \mathbf{v} \cdot a$$

3. Inner Product

$$\mathbf{v} \cdot \mathbf{w} = \mathbf{w} \cdot \mathbf{v}$$

4. Linear Combination

$$\mathbf{v} = c_1\mathbf{e}_1 + c_2\mathbf{e}_2 + \ldots + c_n\mathbf{e}_n$$

Finite Dimensional Vector

```go
type Vector interface {
        Point
        Additive
        Scale(Real) Vector
        Inner(Vector) Real
        Basis() []Vector
}
```

Distance between Two Points

$$d(p,q) = \sqrt{\mathbf{v} \cdot \mathbf{v}}, \quad \mathbf{v} = \vec{pq}$$

```
func Distance(p, q Vector) Real {
	if p.Len() != q.Len() {
		panic("Distance: points have different dimensions")
	}
	v := p.Add(q.AddInv()).(Vector)
	return Sqrt(v.Inner(v))
}
```

Implementation of Vector

- `SimpleVector` is a `Point`.

```go
type SimpleVector []float64

func (p SimpleVector) Equals(q any) bool {
	if qq, ok := q.(gocalc.Point); !ok {
		return false
	} else if len(p) != qq.Len() {
		return false
	} else {
		for i := 0; i < len(p); i++ {
			if !SimpleReal(p[i]).Equals(qq.Map(i)) {
				return false
			}
		}
		return true
	}
}
```

```go
func (p SimpleVector) Len() int {
	return len(p)
}

func (p SimpleVector) Map(i int) gocalc.Real {
	if i < 0 || i >= p.Len() {
		panic("SimplePoint.Map: index out of range")
	}
	return SimpleReal(p[i])
}
```

- `SimpleVector` is an `Additive`.

```
func (p SimpleVector) Add(q gocalc.Additive) gocalc.Additive {
	if qq, ok := q.(gocalc.Point); !ok {
		panic("SimplePoint.Add: incompatible types")
	} else if len(p) != qq.Len() {
		panic("SimplePoint.Add: dimension mismatch")
	} else {
		r := make(SimpleVector, len(p))
		for i := 0; i < len(r); i++ {
			r[i] = p[i] + qq.Map(i).ToFloat()
		}
		return r
	}
}

func (p SimpleVector) Zero() gocalc.Additive { return make(SimpleVector, len(p)) }

func (p SimpleVector) AddInv() gocalc.Additive {
	q := make(SimpleVector, len(p))
	for i := 0; i < len(p); i++ {
		q[i] = -p[i]
	}
	return q
}
```

- `SimpleVector` is a `Vector`.

```
func (p SimpleVector) Scale(k gocalc.Real) gocalc.Vector {
	q := make(SimpleVector, len(p))
	for i := 0; i < len(p); i++ {
		q[i] = k.ToFloat() * p[i]
	}
	return q
}

func (p SimpleVector) Inner(q gocalc.Vector) gocalc.Real {
	if len(p) != qq.Len() {
		panic("SimplePoint.Inner: dimension mismatch")
	} else {
		var s float64
		for i := 0; i < len(p); i++ {
			s += p[i] * qq.Map(i).ToFloat()
		}
		return SimpleReal(s)
	}
}
```

```go
func (p SimpleVector) Basis() []gocalc.Vector {
	basis := make([]gocalc.Vector, p.Len())
	for i := 0; i < p.Len(); i++ {
		basis[i] = make(SimpleVector, p.Len())
		basis[i][i] = 1
	}
	return basis
}
```

This video was produced with the support of CTL in DGIST.

Production: Hyosang Kang

Editing: Jongwoo Baek

본 동영상은 DGIST의 CTL의 지원을 받아 제작되었습니다.

제작: 강효상

편집: 백종우